

AI-Based Restoration of Degraded Images for Semiconductor Inspection

PHASE 2

A 6.81 M parameter MiniRestormer, a channel-attention transformer U-Net that predicts a correction on top of a bicubic upsample, turning a 128×128 speckled input into a 256×256 restoration in a single forward pass.

		PSNR	SSIM	LPIPS
floor	20.380	0.5085	0.4869	
ours	23.706	0.6230	0.1577	
reference	26.579	0.7530	0.3346	

TEAM

Git Push and Pray

Kalyan S | R Kiruthika | Sanjay G
Indian Institute of Technology

REPOSITORY

github.com/KAlcorpse/Git_push_and_pray_KLA

Problem understanding and the restoration task

What we are given

- 4,785 paired images: clean GT 256×256 in [0,1], degraded NoisyLR 128×128
- NoisyLR deliberately escapes [0,1] — measured −0.28 to +2.24
- Speckle, Poisson shot, additive read noise, then downsampling — order not disclosed

What we must produce

- A standalone script: input dir to output dir, same filenames, 2H × 2W
- Two scales: 128→256 and 256→512
- Generalise to unfamiliar image content, not just unfamiliar noise

How it is scored

- PSNR, SSIM and LPIPS — weights undisclosed, and the organisers declined to say
- End-to-end H100 time: faster preferred when quality is comparable
- **Test set has OOD samples from different sources; evaluates accuracy AND robustness**

Our reading of the task: this is a denoising problem wearing a super-resolution costume.

A perfect denoiser followed by plain bicubic scores 26.579 dB, so the 2× upsample costs almost nothing and essentially all the headroom is in removing speckle. That measurement set our priorities before we trained anything.

We closed 55% of the PSNR headroom and 71% of the SSIM headroom.

SUBMITTED 23.706

55% of the available headroom closed

floor 20.380
bicubic on noisy input

reference 26.579
perfect denoise + bicubic

23.706 dB is a SAME-SOURCE number.

The dataset is ordered by source: 23.4% of our held-out images have a near-duplicate in training. Rankings are unaffected; the absolute value is optimistic.

Reference range measured on the same 479 held-out images used for every number in this deck. "Reference" = the GT itself downsampled and bicubic-upsampled: what a flawless denoiser would score. It is not an upper bound.

Dataset analysis and degradation observations

$$\text{Var}(y|x) = \underset{\text{speckle}}{s^2 \cdot x^2} + \underset{\text{shot}}{a \cdot x} + \underset{\text{read}}{g^2}$$

Fitted from real pairs, not assumed. $s = 0.1590$ (Gamma) $a = 0.008018$ (Poisson) $g = 0.0293$

Three terms, not two

Adding the Poisson shot term took the variance fit from

R^2 0.9963 → 0.9995

At dark pixels shot noise DOMINATES — and roughly half of every image is near-black. The old two-term fit absorbed it by inflating the other two.

Speckle is Gamma, not Gaussian

Variance identifies a model only to second moment; the shape was wrong. Real residuals are skewed and heavy-tailed at low intensity. Gamma carries the right positive skew — the standard coherent-speckle model.

Noise splits across the resize

Real residual autocorrelation is -0.051 .

All noise after resize → -0.000 (too white). All before → -0.155 (3× too correlated).

VAR_AT_HR = 0.36 reproduces it (-0.055). Downsampler identified as bicubic, no AA, from 10 candidates.

Level varies per image

$p_{90}/p_{10} = 1.30\times$

(sd 0.113, full range 3.65×)

One set of weights must average over that spread — the argument for noise conditioning.

Data audit: the low-PSNR tail is REAL, not junk

Our worst images look like salt-and-pepper. Measuring lag-1 autocorrelation of the GT: across all 4,785 images **exactly three are genuinely degenerate**.

Everything else at low autocorrelation is real, very fine-grained SEM — the hardest end of the distribution, and the texture we already under-reproduce. ! Deleting it would remove the strongest signal against over-smoothing.

Known remaining defect

The dark decile is still too light-tailed: **kurtosis 0.59 against a real 2.55**.

Something at low signal is unmodelled. This is why synthesis still loses to real pairs, and it is where to look if anyone retries.

End-to-end pipeline

01

INPUT

Scans the directory and groups .npy files by shape. Mixed 128/256 inputs batch safely.

02

READ

np.load → float32, stacked to (B,1,H,W). Threads start BEFORE import torch.

03

HOST → GPU

channels_last, non-blocking. Batch auto-probed from free VRAM.

04

RESTORATION

6.81 M model, fp16 autocast, TF32. 128²→256², single forward pass.

05

GPU → HOST

Explicit CUDA synchronise so real transfer latency is measured.

06

SAVE

np.save float32 at 2× resolution. Same filename, format never changes.

EVERY STAGE IS IN THE TIMED WINDOW

	479 images, RTX 3050, fresh process
import+init	1.29 s <- irreducible
build model	0.38 s
inference	12.22 s
TOTAL	13.88 s

Startup dominates on an H100

import torch is 1.50 s and CUDA context 0.30 s — **~1.8 s no model change can touch.**

If inference falls to 0.5–1.4 s on an H100, startup is **56–78% of end-to-end.**

So shrinking the model for speed is dead: it buys ~10% of wall time and costs real quality.

Single forward pass — no TTA

8× dihedral re-measured: +0.086 dB PSNR but **LPIPS degrades 39%**, for 8.8× the compute. Rejected.

No ensemble — one model

Averaging our three 36-epoch models moved every metric **inside the 0.006 dB seed floor, for 2–3× the cost.** Rejected.

cuda.benchmark is now ADAPTIVE

Autotuning finds 11%-faster kernels but costs 26.5 s up front — it repays only at **~9,650 images.** Chosen from the input count at runtime.

Preprocessing, paired loader & the full-res phase

Paired loader and 64px crops

Each sample takes a random 64×64 window of the 128×128 input and the exactly corresponding 128×128 window of the GT — strict 2× correspondence.

65×65 positions × 8 D₄ transforms ≈ **34k views per image, and it fits batch 6 on 4 GB.**

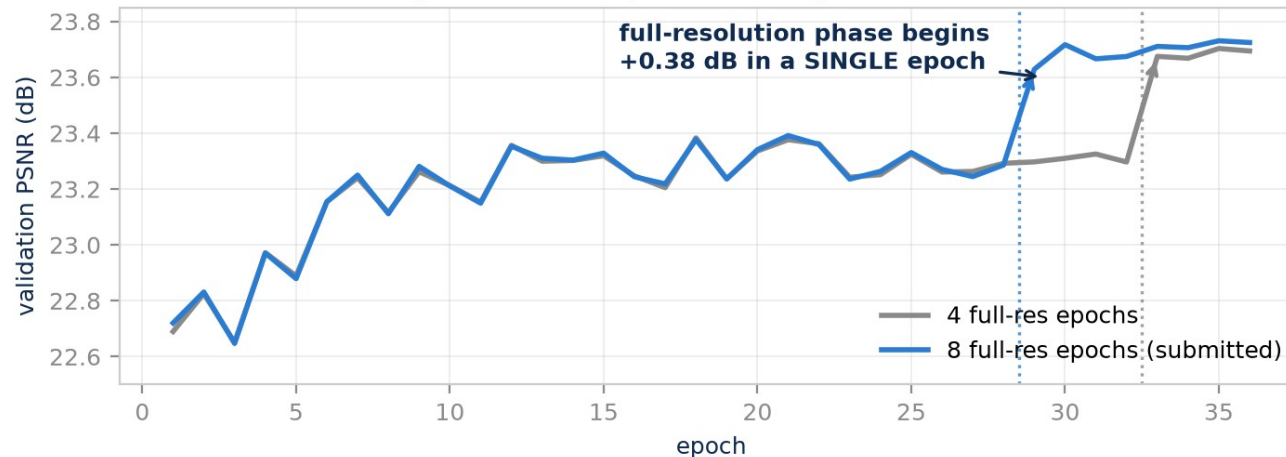
The full-resolution phase — the biggest single win in the project

The last FULLRES_EPOCHS epochs drop the crop and train on whole 128→256 images.

+0.377 dB / +0.0127 SSIM / -0.0110 LPIPS — the only change that moved all three metrics the same way.

Why it is large: training on crops runs the bottleneck at 16×16 while inference runs it at 32×32, and 71% of the parameters live at that level. The phase closes a train/test gap, it is not just extra training.

The full-resolution phase is a step, not a ramp



It is a STEP, not a ramp

Almost the whole effect lands in **one epoch**. Epochs 5–8 of the phase add only **+0.046 dB** at ~9 min each.

So FULLRES_EPOCHS=12 was ruled out by measurement rather than by burning a run on it.

Scale: measured, and the news is good

Content held fixed, only inference size varied. True size sensitivity **0.264 dB** across 4×, and it **FAVOURS** larger inputs. 128 beats 64 even though the model trained on 64 crops for 28 of 36 epochs, so 256→512 is low risk.

Augmentation

Two different things share the word

Geometric (always on, since run one): 4 rotations × 2 flips × random crop. Exact pixel permutations, no interpolation, so 2× correspondence and speckle statistics stay intact.

Degradation (new in Phase 2): varies HOW NOISY the input is. Geometric augmentation does not — a rotated noisy image is just as noisy.

Falsified, reversed, then retracted

A stress test appeared to reverse an earlier negative verdict — until we noticed **stress.py** and **augment.py** import the same constants from **degrade.py** and apply them identically.

The probe perturbs images with exactly the distribution the augmented models were trained on. It measures robustness to a self-generated perturbation, not to anything out of distribution.

Variety of KIND, not of AMOUNT

Re-run honestly: train on a **diverse** family — Gaussian speckle instead of Gamma, plus a blur the forward model has never contained — and test on a **held-out** family no augmenter produces.

LPIPS degrades 3.4× less on degradations never seen (+0.124 vs +0.421).

The control settles it: scaling our own fitted noise gives +0.418 — no benefit at all, on identical budget and augmentation probability.

Cost: +0.0072 LPIPS in-distribution, 14× the seed floor. A real trade, not a free win.

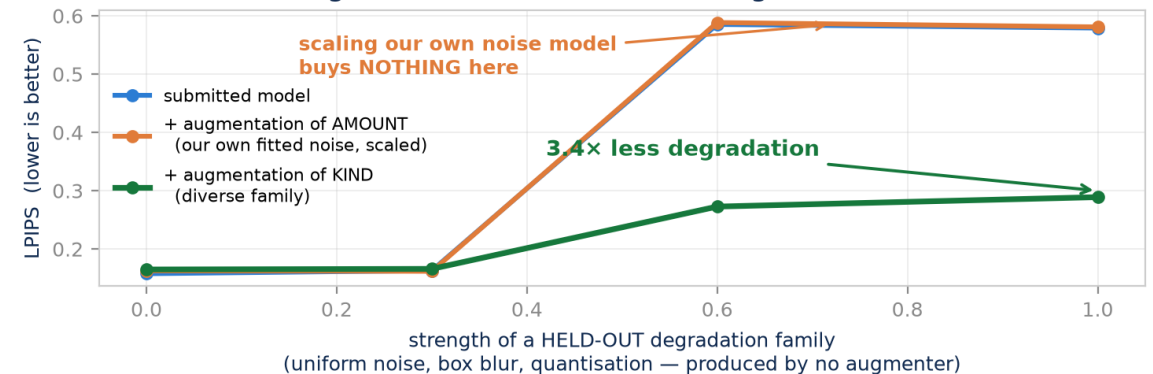
Synthetic pairs from GT — rejected

Replacing the input with one synthesised from GT, across three successively better forward models:

−0.048 dB / −0.0063 SSIM / +0.0092 LPIPS vs a matched control, all CIs excluding zero.

The model got measurably better each round and the sign never flipped.

Robustness to degradations never seen in training

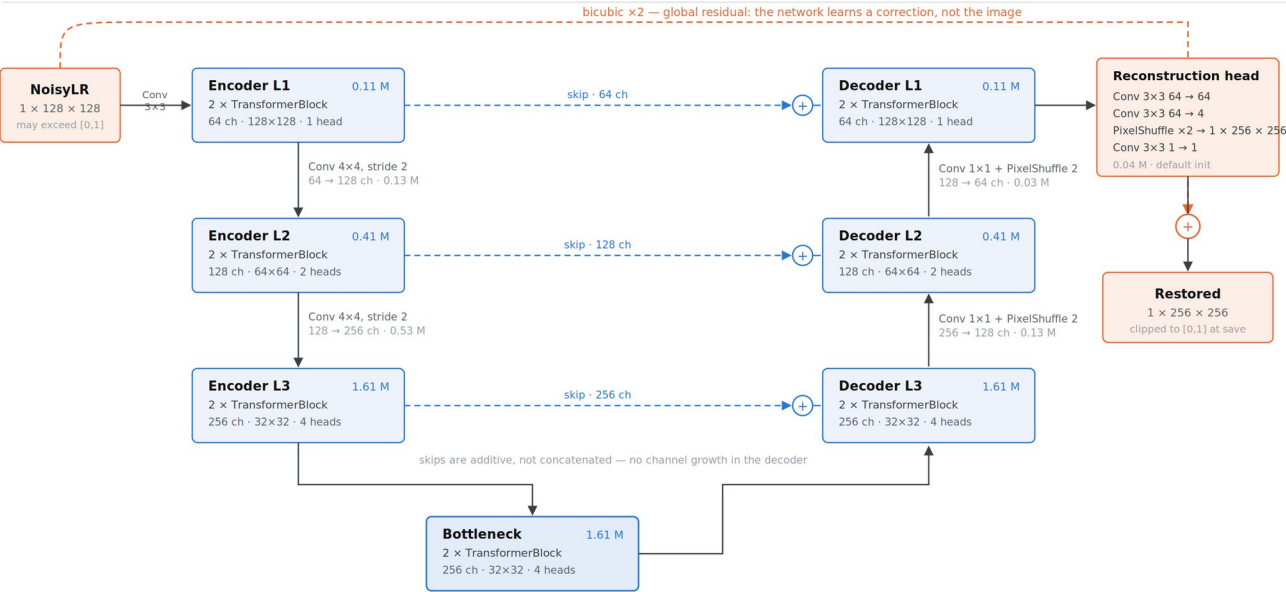


Rejected augmentations

Brightness / contrast — changes signal-dependent speckle variance · rotation / resize / elastic — interpolation breaks the exact 2× relation · mixup / cutmix — destroys pixel-for-pixel LR-to-GT correspondence

Model architecture and design rationale

MiniRestormer — 6.73 M parameters, $1 \times 128 \times 128 \rightarrow 1 \times 256 \times 256$
dim 64 · blocks (2, 2, 2) · heads (1, 2, 4) · additive encoder-decoder skips · 2x PixelShuffle · global bicubic residual · fp32



stage	params	share
encoder blocks	2.130M	31.3%
decoder blocks	2.130M	31.3%
bottleneck	1.609M	23.6%
down / up sampling	0.821M	12.0%
in conv, skip, head	0.121M	1.8%
TOTAL	6.810M	100%

The supplied diagram is an earlier revision showing 6.73 M. The shipped model is **6.810 M** — the head is **0.121 M**, not 0.04 M. Budget above is measured from weights.pth.

TransformerBlock — the unit repeated 2x at every level

Parameter budget

Bicubic residual skip

The network predicts a correction, not 65k pixels from scratch, so training starts near a strong baseline.

Three modes measured: learned > bicubic > none — but the whole spread is ≤ 0.07 dB.

Attention over channels

Cost is linear in H·W and no layer has a fixed spatial size, so a **128-trained model runs at 256 or 512 unchanged** — directly why two scales are safe.

Additive skips, wide head

Skips add rather than concatenate, keeping the decoder at encoder width — where the saving over full Restormer comes from. The 32-channel head removed a 1-channel bottleneck.

The learned skip barely moved

After 36 epochs it drifted only **5.25% from its bicubic initialisation (max tap change 0.027)**.

So learned $\approx$ bicubic in the metrics because it is still $\approx$ bicubic in the weights.

Loss functions and training setup

$$L = 0.50 \cdot \text{Charbonnier} + 0.35 \cdot (1 - \text{SSIM}) + 0.15 \cdot \text{gradient} + 0.10 \cdot \text{LPIPS}$$

Checkpoint selection uses **SSIM – LPIPS**, never PSNR

The four terms

Charbonnier 0.50 — smooth L1, robust to the speckle outliers that make plain L2 chase bright pixels.

1–SSIM 0.35 — structural, but it REWARDS BLUR: blurring bicubic raised its SSIM 0.831→0.887. Deliberately down-weighted from 0.5.

gradient 0.15 — L1 on Sobel, pushes back against both blurring terms.

LPIPS 0.10 — the main force against over-smoothing.

SSIM is already 60% of the loss

Measured on the shipped model's real predictions:

W_CHAR·charbonnier 0.030 12.0%

W_SSIM·(1–SSIM) 0.151 **60.4%**

W_GRAD·gradient 0.069 27.7%

So 'prioritise SSIM' is probably not about raising W_SSIM — the term already dominates, and raising it further buys blur. The question is whether it is the RIGHT SSIM.

We were optimising a different SSIM

Our loss used an **11×11 Gaussian**; score.py reports skimage's **7×7 uniform**. They differ by **0.024 SSIM** — ~48× the seed floor — though they correlate at 0.9966.

Trained on the exact scored metric: **+0.0005 SSIM for –0.050 dB PSNR**.

Rejected — the SSIM gain IS the seed floor; the PSNR loss is 8× it.

Frequency loss: the diagnostic is not the goal

L1 on $\log_{10}|FFT|$ lifted high-frequency retention **0.470 → 0.589** — the largest move on that quantity in the project — and **LPIPS did not change** (CI spans zero), for 0.111 dB of PSNR.

hf_ratio measures how MUCH high-frequency energy there is, not whether it is the RIGHT energy. Invented texture earns nothing from a perceptual metric.

A fourth, UNSCORED diagnostic: hf_ratio, high-frequency energy above 0.25 cycles/px relative to GT. Its only job is to catch the blur that both headline metrics reward.

```

optimiser  AdamW, weight decay 1e-4
schedule   3-epoch warmup -> cosine to 1e-6
LR / batch 1e-4 / 6 (auto-probed)
precision  fp32 training, fp16 autocast at inference
stabilisers EMA 0.999, grad clip 1.0, spike reject
split      90/10, seed 42, held out from training
           AND from model selection
schedule   EPOCHS=36, FULLRES_EPOCHS=8
  
```


Experiment tracking and baseline comparison

The confound that invalidated our own headline

train.py gates the full-res phase on epoch > EPOCHS — FULLRES_EPOCHS. A run capped by MAX_HOURS leaves EPOCHS=400, so the condition reads epoch > 396 and **the phase never fires**.

We compared one of each and credited **+0.33 dB** to synthetic data. The correct control showed synthetic data **HURTS by 0.048 dB** — and the mis-attributed gain was the full-res phase, which became the project's biggest win.

The biggest result we have come out of chasing down our own mistake.

one variable per run		PSNR	SSIM	verdict
full-res phase fires	+0.377	+0.0127	+BIGGEST WIN	
FULLRES_EPOCHS 4 -> 8	+0.038	+0.0023	+adopted	
EPOCHS 11 -> 36	+0.059	+0.0045	+adopted	
SKIP learned vs bicubic	+0.002	+0.0002	not measurable	
synthetic data 25%	-0.048	-0.0063	rejected	
NOISE_COND alone	-0.007	-0.0030	rejected	
degradation augmentation	+0.015	-0.0007	robustness win	
TTA 8x	+0.086	+0.0038	LPIPS +39%	
ensemble of 3 models	+0.008	-0.0003	inside noise	

We measured the noise floor

Two runs identical but for SEED:

converged final checkpoint **0.006 dB**

mid-training **0.388 dB**

same-seed (cudnn/TF32) 0.010 dB

Rule: only compare fully-annealed final checkpoints. A truncated or mid-trajectory number is worth nothing — two seeds can be 0.388 dB apart.

Ten corrections we made to our own claims

Almost every one was a variable we assumed constant across a comparison.

- 'three-way tie' — unpaired means hid a real ordering
- pure-Charbonnier '+0.43 dB' — LPIPS more than doubled; the log read 0.00000 only because the LPIPS net never initialised
- synthetic data '+0.33 dB' — schedule confound
- 'ceiling is 24.67 dB' — wrong image subset, and it is not a bound
- 'every run used the wrong skip mode' — overstated, not measurable at 11 epochs
- 'augmentation is a loss' — measured on the wrong axis, later reversed

Every delta in this deck comes with a paired bootstrap 95% CI. Pairing removes image-to-image variance, which dominates the raw spread — our first architecture comparison looked like a three-way tie until we paired it.

PSNR, SSIM and LPIPS results

Every challenger against the shipped model — 479 held-out images

	run	PSNR	SSIM	LPIPS	vs SUBMITTED (95% CI)		
SUBMITTED	23.706	0.6230	0.1577	--			
aug+cond	23.737	0.6219	0.1625	+0.031*	-0.0011*	+0.0048*	
aug	23.721	0.6223	0.1608	+0.015*	-0.0007*	+0.0030*	
4 full-res	23.668	0.6207	0.1599	-0.038*	-0.0023*	+0.0022*	
cond	23.662	0.6200	0.1602	-0.045*	-0.0030*	+0.0025*	
11 epochs	23.607	0.6160	0.1737	-0.099*	-0.0070*	+0.0160*	
Phase 1 arch	23.306	0.6120	0.1643	-0.362*	-0.0087*	+0.0043*	

* = CI excludes zero. seed noise floor = 0.006 dB

Why we ship this one

It is the only model that is **best on all three metrics at once**.

With the metric weights undisclosed, that is the only robust position to hold — every challenger wins one metric by trading another.

The augmented + conditioned variant wins PSNR by 0.031 dB but costs 0.0048 LPIPS — 10× the LPIPS seed floor.

The open submission decision

The augmented + conditioned variant is a hair worse on two metrics in-distribution and **better on ALL THREE the moment the input is noisier than training: +0.322 dB at t=0.6, +0.691 dB and -0.046 LPIPS at t=1.0**.

The brief says the test set includes OOD samples and evaluates robustness. **This is a genuine decision, and it turns on how much of the test set is OOD —**

Robustness measured on three axes

content / source fine — corr(familiarity, PSNR) = +0.042

scale fine — 0.264 dB over 4×, favouring larger inputs

noise level WEAK — still 1.0 dB worse than bicubic given a clean input

Two of the three axes the organisers will test measure fine. The weak one is noise level.

Two suggestions from mentoring, both measured, both rejected: matching the loss's SSIM exactly to the scored metric (+0.0005 SSIM, which IS the seed floor, for -0.050 dB), and a frequency-domain loss (+25% high-frequency retention, no LPIPS change, -0.111 dB). Each was scored in-distribution AND on both OOD axes, so neither verdict rests on a single measurement.

Runtime, batch size and optimisation

13.1 s

end-to-end for 479 images, measured start to finish on a 4 GB dev GPU. No TTA, no ensemble.

```
python start    0.02 s
import torch    1.50 s <-- dominates
CUDA context    0.30 s
model build     0.09 s
IRREDUCIBLE    ~1.8 s
```

What that means on an H100

If inference falls to 0.5–1.4 s, startup is **56–78% of end-to-end**.

Perfect kernel optimisation could recover at most ~40%; a realistic 2× from INT8 recovers 15–20%.

So we spend inference time on quality, not save it.

cudnn.benchmark sweep, one fresh process per row			
N	off	on	on/off
128	3.66	29.80	8.14x
479	12.70	42.69	3.36x
1916	48.36	63.10	1.30x
5748	140.66	153.16	1.09x

off = 1.04s + 24.33 ms/img on = 27.6s + 21.58 ms/img
crossover ~9,650 images -> BENCHMARK_MIN_IMAGES = 10000

Compute-bound, so no launch-overhead headroom

ms/img is **flat from batch 1 to 16** and the CPU→GPU launch gap is ~0: a **single 128×128 image already saturates a 20 W 3050**.

CUDA graphs and kernel fusion would gain nothing here. On an H100 this inverts — it will be launch-bound at small batch, and **auto_batch's cap of 256 is the one speed parameter we are genuinely flying blind on**.

No torch.compile

30–60 s of compile time to save well under 1 s of compute. Ruled out by arithmetic, not tried and abandoned.

Nothing heavy is imported

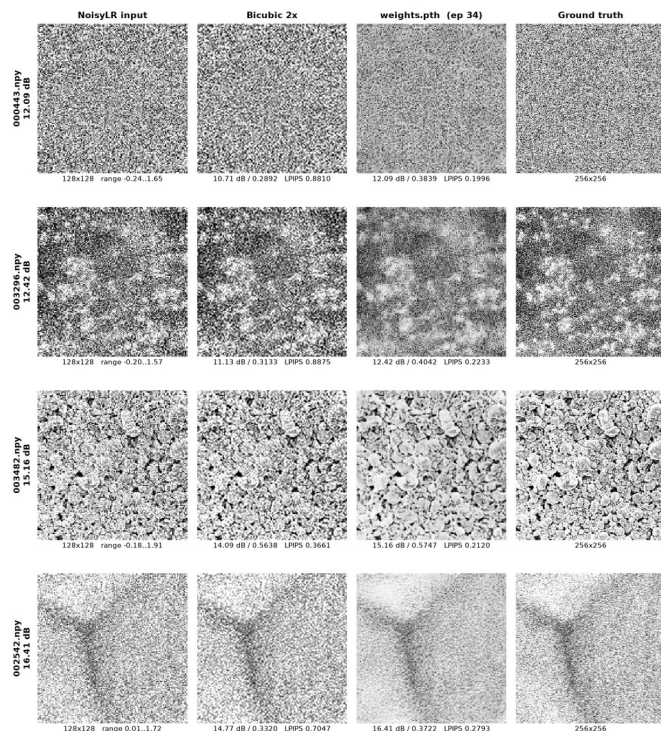
No torchvision, lpips or skimage in the inference path — they cost 1–2 s of startup and are training and scoring dependencies only.

Resolution-agnostic by construction

Inputs are reflect-padded to a multiple of 8 and cropped back; images are grouped by shape so 128 and 256 inputs never share a padded batch.

Visual results, failure cases and limitations

Held-out validation — worst 4 of 479



The worst cases are not a bug

Our four lowest-PSNR images look like salt-and-pepper. Measuring lag-1 autocorrelation of the GT, only **one of the four is genuinely near-noise** — and **dataset-wide only 3 of 4,785**.

The rest is real, very fine-grained SEM. **We still beat bicubic on those images by 1.38 dB and LPIPS 0.1996 vs 0.8810. This is the hard end of the real distribution, not junk to delete.**

Where it fails

When speckle destroys a texture completely, the loss-optimal answer is the average of every texture that could have been there — and that average is a blur. A fundamental limitation of supervised learning on this data, not a defect in the code.

Dropping the worst 24 val images moves the mean 23.706 → 24.054 dB, so a large part of any score is decided by images nobody can restore.

Honest limitations

We retain only ~47% of GT's high-frequency energy.

Given a CLEAN input the model scores 1.0 dB WORSE than plain bicubic — it applies one fixed denoising strength and cannot back off. Four runs across two queues moved that by 0.09 dB. It is our largest known weakness.

23.706 dB is a same-source number: 23.4% of held-out images have a near-duplicate in training.

Conclusion, external resources and repository

What we are submitting

- A single 6.81 M-parameter MiniRestormer, one forward pass per image: **23.706 dB / 0.6230 SSIM / 0.1577 LPIPS on 479 images held out from training AND from model selection**
- 55% of the measured PSNR headroom and 71% of the SSIM headroom, against a floor and a reference we measured rather than assumed
- The only configuration that is best on all three metrics at once — chosen over a higher-PSNR variant and over a more noise-robust variant, in both cases because the metric weights are undisclosed
- 13.1 s end-to-end for 479 images, of which ~1.8 s is irreducible process startup

Next steps, ranked and costed

1. LOG_INPUT — never tested, and Phase 1 named it. Speckle is multiplicative, so a log channel makes it additive and signal-independent. ~2 h
2. A texture-robust noise estimator — the current one is 58% texture and correlates only $r=0.74$ with the true level, so conditioning has been fed a signal wired backward on exactly the images that matter
3. CROP=128 — the remaining lever on size sensitivity, now that extending the full-res phase is spent. ~2.5 h
4. Decide SUBMITTED vs aug+cond once we know how much of the test set is OOD

EXTERNAL RESOURCE DISCLOSURE

arxiv.org/abs/2111.09881 Restormer: MDTA / GDFN blocks
arxiv.org/abs/2204.04676 NAFNet: TinyNAFNet baseline
 LPIPS (Zhang 2018, AlexNet) training loss + reported
 metric. NOT in the inference path.
 PyTorch, NumPy, scikit-image, SciPy framework, metrics

Reproducibility

Seed 42 · 90/10 split held out from training and selection · every hyper-parameter overridable by environment variable, so no experiment required a source edit · per-epoch train_log.csv for every run · the checkpoint's full architecture is recovered from its weight shapes at load time, so inference cannot silently build the wrong model.

REPOSITORY github.com/KAlcorpse/Git_push_and_pray_KLA